FreeDOS

Apuntes del profesor

Material reorganizado a partir del PDF original proporcionado. Se conserva el contenido, los ejemplos y los procedimientos del documento; la presentación web funciona como apoyo visual.

Recorrido de la clase

1. Antecedentes históricos y propósito de FreeDOS
2. Disco, archivos y sistemas de archivos
3. Secuencia de arranque y firmware
4. Kernel, componentes y modelo de ejecución
5. Arquitectura x86, registros y memoria
7. Casos de uso, vigencia y conclusiones
8. Introducción: virtualización y emulación
9. Práctica: preparar el entorno DOSBox-X
10. Práctica: programa con DEBUG
11. Práctica: ensamblador NASM
12. Práctica: OpenGEM

Criterio de edición

Se reordenaron los bloques para seguir una progresión: historia y propósito; almacenamiento; arranque; componentes y arquitectura; memoria e interrupciones; vigencia; y finalmente las prácticas. Las formulaciones originales se mantienen como material de base. Las imprecisiones técnicas no se sustituyen silenciosamente: deben señalarse al explicarlas.

De DOS a FreeDOS: contexto histórico

Este bloque se incorpora antes del desarrollo que ya estaba en los apuntes. Su función es ofrecer el marco temporal y explicar por qué tiene sentido estudiar FreeDOS. El resto del material conserva su orden y contenido.

1. Los sistemas DOS y las primeras PC

El material original sitúa a la familia DOS en las computadoras personales de las décadas de 1970 y 1980, especialmente en equipos basados en los procesadores Intel 8086/8088. Su interfaz habitual era textual y el usuario interactuaba escribiendo órdenes.

2. 1981: MS-DOS y el IBM PC

Según los apuntes originales, MS-DOS aparece en 1981 asociado al IBM PC y se convierte en el sistema de referencia de la industria. También existieron alternativas como PC-DOS y DR-DOS. Durante los años siguientes, DOS ocupó un lugar central en las computadoras personales compatibles.

3. El cambio de escenario durante los años noventa

La expansión de Windows modificó progresivamente el papel de DOS. Los apuntes señalan a Windows 95 como un punto de inflexión: la interfaz gráfica dejó de depender del arranque manual de Windows desde el entorno DOS como ocurría en versiones anteriores.

4. Junio de 1994: nace la propuesta de FreeDOS

En junio de 1994, Jim Hall propuso crear un sistema compatible con MS-DOS, libre y de código abierto. La idea era que una comunidad de desarrolladores pudiera mantenerlo y darle continuidad. FreeDOS conserva así un entorno DOS utilizable para compatibilidad, sistemas heredados, retrocomputación y enseñanza.

Transición al contenido técnico

Con este contexto presentado, la clase continúa con la diapositiva “¿Qué es DOS?”. A partir de allí se mantiene exactamente el recorrido existente: definición y características de DOS; relación entre MS-DOS y FreeDOS; discos y FAT; arranque; componentes; procesador y memoria; interrupciones; vigencia; y prácticas.

Nota de fidelidad: esta introducción se construyó con los datos históricos presentes en el PDF original. No se añadieron antecedentes externos no desarrollados en ese material.

1. Antecedentes históricos y propósito de FreeDOS

Antecedentes Históricos:

¿Qué significa DOS?

Disk Operación System. Son una familia de sistemas operativos diseñados sobre la arquitectura de los procesadores Intel 8086/8088, para los ordenadores personales de las décadas de los 70/80.

Características:
- Modo texto y CLI: En modo nativo no existe interfaz grafica, la interacción entre usuario y el sistema se da a travéz de una Command Line Interface(aplicación de terminal o comandos en consolas);
- Monotarea y monousuario: un solo programa utiliza la totalidad del procesador(no hay hilos), y tampoco existen privilegios o características diferenciadas para usuarios, ni múltiples usuarios;
- Acceso directo al hardware, en este concreto a la memoria, situación que no se da en ningún sistema operativo moderno por cuestiones de seguridad, solo se obtiene una característica mediante la api del sistema operativo;
- Arquitectura de procesador de 16bits con memoria de 640kb + 384kb. Esto quiere decir que si bien la memoria llegaba a un 1mb, el procesador solo podía mover 16bits por vez(por ciclo), donde busca, lee, procesa y devuelve, desde y hacia la memoria.

MS-DOS:

Fue la estándar de la industria, pese a que existían competidores como PC-DOS o DR-DOS, nace en 1981 para la computadora IB M PC, domino el mercado de sistemas operativos hasta principio de lo 90. Sin embargo en 1995, Windows 95 le iba a dar el golpe de gracia a este sistema, porque evolucionaria hacia una arquitectura de 32bits y se convertiría en multitarea. Ademas windows ya no lo necesitaba para lanzarse, antes de la version 95, el sistema operativo por defecto era ms-dos, es decir lo primero que veías en pantalla y con el comando win, recién iniciaba la interfaz grafica, esto hacia que ms-dos fuera el lanzador oficial de windows.

En junio de 1994 Jim Hall se propuso crear un sistema operativo compatible con MS-DOS, pero de dominio publico y de codigo abierto. La finalidad de esta idea es que una vez que un masa critica de desarrolladores lo adopte sea la misma comunidad la que le brinde soporte y mantenimiento, además de que las licencias libres como la GNU GPL permiten lucrar con el. (en procesador de x86 podemos usar freedos para crear un punto de venta, un viejo sistema de pagos, un firmware para BIOS, un cajero automático, etc).

¿Para que se usa hoy?

- Retrocomputacion y gaming(arcades)
- Entornos industriales(viejas PLC's)
- Sistemas embebidos
- Distribución de hardware sin SO(Lenovo, HP)
- La principal que es el fin historio y educativo, donde el alumnos el desarrollo de la computación del sistemas iniciales, el movimiento de los datos dentro de un circuito informático y puede usarlo libremente para aprender.

2. Disco, archivos y sistemas de archivos

Sistema de archivos:

Un disco es solo almacenamiento, bruto e ilegible, un sistema magnético que graba o quema información. Para poder ordenamiento dicha información establece un conjunto reglas o protocolos para ordenarla. Sin ello el sistema no sabe donde empieza o donde termina un archivo. Para el procesador un archivos son solamente bytes.

Un FileSystem tiene dos componentes, un índice y un conjunto de reglas de organización. Estas reglas tienen 3 elementos, nombre(de archivo o directorio), locación(el segmento de disco donde esta alojado

o clúster) y seguridad(permisos de acceso y status de usuario).

Tipos de FileSystem:

1- FAT(file allocation table): es uno de los sistema mas viejos, creado para MS-DOS(Shell, SO, interfaz),

hasta Windows 3. Luego, a partir de Windows 95, el so se independiza de su lanzador.
- Utiliza una tabla principal que se carga al principio del disco, y funciona como índice, esta registrado el o los fragmentos que se vinculan a un archivo a través de punteros.

2- FAT16 vs FAT32:
Las mayores diferencias radican en los sectores de direccionamiento y el tamaño de estos:

FAT16, usa sectores de 16 bits con un máximo de dos 2GB para un archivo.
FAT32, usa sectores de 32 bits(28 efectivos) con un máximo por archivo efectivo de 4GB(teóricamente debería ser 16GB)
Se creo ExFAT para superar esos 4GB, pero actualmente tanto FAT32 como ExFAT son usadas solo en dispositivos de almacenamiento portátil(USB hasta 16GB)

3- NFTS(New Tecnología File System):
Desarrollado por Microsoft a mediados de los 90, con el objetivo de brindar seguridad, estabilidad y soporte para archivos mas grandes.
Aquí encontramos todas las características modernas de los sistemas de archivos, a saber:
- Journaling o sistema de registro previo, anota los cambios en disco antes de ejecutar, esto permite recuperar procesos, servicios o estados;
- Permisos de seguridad avanzados(ACL - Access Control List): Lista compuestas por filas llamadas ACE que tienen un sujeto(usuario), un permiso(la acción) y un tipo(permite/niega)
- Soporte archivos hasta 16TB y tamaños de partición de hasta 8TB.
- EFS: Compresión y cifrado de directorios;

4- EXT(Extended File System): es el estándar desarrollado para sistemas Linux y Unix.
Tenemos 4 versiones:
- EXT1 y 2: rápido y robusto pero sin Journaling;
- EXT3: introdujo Journaling
- EXT4: estándar actual que soporte volúmenes de hasta un 1Exabyte y archivos hasta 16TB(Equivale exactamente a 1.000.000.000.000.000.000 de bytes (un trillón de bytes en la escala corta, o un millón de terabytes))

3. Secuencia de arranque y firmware

Conceptos Básicos:

1. BIOS / UEFI \(\longrightarrow \) El Despertar del Hardware
• ¿Qué pasa?: El procesador ejecuta el firmware de fábrica de la placa madre. Se realiza el POST (Power-On Self-Test) para verificar el estado de los componentes físicos (RAM, procesador, etc.).
• El rol de la IVT: Durante este mismo paso, la BIOS escribe de forma obligatoria la IVT original en el primer Kilobyte de la memoria RAM (direcciones 00000h a 003FFh). La llena con las funciones de su propio firmware para el control básico de la pantalla y el teclado.

2. Gestor de Arranque (GRUB / MBR) \(\longrightarrow \) La Búsqueda del Sistema
• ¿Qué pasa?: La BIOS lee el Registro Maestro de Arranque (MBR) o el Registro de Arranque de Volumen (VBR) del disco duro utilizando las interrupciones de disco de la IVT que acaba de crear. El gestor de arranque (como GRUB) se carga en la memoria y localiza el archivo ejecutable del sistema operativo.

3. Kernel \(\longrightarrow \) El Núcleo Toma el Control (KERNEL.SYS)
• ¿Qué pasa?: El gestor de arranque copia el archivo KERNEL.SYS (el núcleo de FreeDOS) en la memoria RAM y le cede el control absoluto [Explore the five steps of the FreeDOS boot

sequence]. El Kernel estabiliza el sistema y toma el mapa de la IVT heredado de la BIOS para actualizarlo con las rutinas nativas del sistema operativo.

4. Archivo de Configuración \(\longrightarrow \) CONFIG.SYS (o FDCONFIG.SYS)
• ¿Qué pasa?: El Kernel lee este archivo de texto para configurar el entorno del sistema [How FreeDOS boots up]. Aquí se definen los parámetros de memoria y se cargan los controladores (drivers) de dispositivos de bajo nivel (como gestores de memoria o soporte de archivos).
• El rol de la IVT: Cada controlador que se ejecuta en este paso se aloja en un espacio protegido de la RAM y reemplaza o actualiza las direcciones de la IVT para que el sistema operativo sepa comunicarse de forma avanzada con los periféricos.

5. Intérprete de Comandos y Scripts de Inicio \(\longrightarrow \) COMMAND.COM y AUTOEXEC.BAT

• ¿Qué pasa?: El Kernel lanza el intérprete de comandos (COMMAND.COM), el cual procesa de forma automatizada el archivo por lotes AUTOEXEC.BAT [How FreeDOS boots up]. En este archivo se configuran variables de entorno (como el PATH), el idioma del teclado y se ejecutan los programas TSR del usuario.
• El rol de la IVT: Los programas TSR (como el controlador avanzado del mouse CTMOUSE.EXE o utilidades de captura de pantalla) se quedan "viviendo" de forma pasiva en la RAM y se inyectan en las entradas correspondientes de la IVT para activarse cada vez que ocurra su interrupción.

6. Sistema Operativo Listo

• ¿Qué pasa?: El archivo AUTOEXEC.BAT termina de ejecutarse y la pantalla negra de FreeDOS muestra el indicador de línea de comandos (C:\>) [How FreeDOS boots up]. El sistema está listo esperando las órdenes del usuario, con el Kernel coordinando todo el tráfico a través del mapa definitivo de la IVT en la memoria RAM.

4. Kernel, componentes y modelo de ejecución

1- ¿Qué es un kernel?

El Kernel es el software oculto que traduce el mundo abstracto de tus aplicaciones al mundo físico de los electrones, cables y memorias, asegurándose de que el hardware responda de forma ordenada y segura.

Principio rector del funcionamiento de una computadora: un programa no tiene permitido trabajar sobre los elementos o componentes del hardware, se establece un mediador, un orquestador que sirve de canal entre la instrucción y el proceso. Esto es el kernel.

Funciones:
A- Manejar el procesador(gestionar tareas);
B- Administra memoria: asigna los segmentos de memoria a cada programa, y vigila que un programa no invada la memoria de otro;
C- Hablar con los componentes, a través de drivers o controladores(traductor al idioma del componente distinto del firmware): por ejemplo con placa de audio, adaptador de red, GPU, etc. Los drivers puede ser del propio kernel o propietarios del hardware.
D- Aislamiento y Seguridad, es consecuencia del punto anterior.

Existen tres tipos de kernel:

1- Monolítico: básicamente, funciones de memoria y procesador, y drivers están todos juntos en el mismo lugar, pero si algo falla, falla todo(Linux, Android y freeDOS).
2- Microkernel: se reduce al mínimo posible, solo maneja memoria y procesador, los drivers se ejecutan como programas comunes.
3- Hibrido: su base en un Microkernel pero con autorización para guardar lo que necesite dentro del bloque principal.

FreeDOS se compone de tres grandes componentes, el kernel, el interprete de comandos(freeCOM) y un conjunto de utilidades CLI como ser edit, fdisk, mem, etc. Cabe aclarar que el mantenimiento de cada componente, hoy en día, son proyectos distintos pero regidos sobre el mismo espíritu que plazo en 1994.

FreeDOS es un sistema operativo antiguo, pero como las computadoras modernas (Intel Core o AMD Ryzen) todavía usan el mismo "idioma" básico de fábrica (llamado x86 o CISC) que usaban las computadoras de hace 30 años, FreeDOS puede seguir funcionando en ellas sin problemas, pero depende de la UEFI, que es el sistema de arranque reemplazo a la BIOS tradicional.

En computadoras muy nuevas vamos necesitar, o bien activar el modo legacy o bien usar un hipervisor o emulador según el tipo de instrucción que tenga disponible el procesador.

Modo real de 16 bits, ¿Qué significa?

Los 16 bits son el tamaño de la operación que puede ejecutar el procesador en un ciclo de

procesamiento; mientras más grande es la caja, más información puede manipular de una sola vez. El ciclo de procesamiento está medido por el impulso eléctrico de un cristal de cuarzo, sincronizado al reloj del sistema.

Significa que el procesador opera con registros de 16 bits y carece de mecanismos de protección de memoria, lo cual nos permite cambiar el desplazamiento de memoria del propio sistema operativo (direccionamiento de memoria).

Si bien el procesador solo puede mover de a 16 bits, el cableado que sale a la placa madre o bus de direcciones exige 20 bits (porque tiene 20 patitas que necesitan electricidad) para activar el chip (de la RAM). Entonces, toma un registro normal de segmento, lo estira 4 posiciones a la izquierda y le suma el desplazamiento para generar la dirección final.

Multiusuario y multitarea:
únicamente puede existir un programa en primer plano
ejecutándose en un momento dado, sin planificador de procesos, sin espacios de memoria aislados entre
aplicaciones y sin gestión de usuarios o permisos de acceso a nivel de sistema de archivos. El sistema operativo actúa fundamentalmente como una capa de servicios sobre la cual el programa en ejecución tiene control casi total del hardware disponible, incluyendo acceso directo a los puertos de entrada y salida, a la memoria de vídeo y a los dispositivos periféricos.

Programas TSR:

Como consecuencia la ausencia de un mecanismo multitarea, se procede a simular ejecución concurrente mediante los programas "Terminate and Stay Resident". Es un programa que al finalizar su ejecución no libera memoria, sino que queda cargado en forma pasiva, esperando un interrupción o un evento que lo libere.

Fue mediante esta implementación que se pudieran agregar servicios en segundo plano, como controladores de mouse, gestores de discos, captura de pantalla, etc.

Para que esto funcione lo primero que carga el kernel es la IVT(interruptions vector tables) o tabla de vectores de interrupciones, hay es donde viven procesos ya ejecutados a la espera de una interrupcion para volver a ejecutarse.

Momento 1: La BIOS/UEFI llena la IVT con lo básico (Firmware)
Cuando la computadora apenas enciende, en esa IVT solo hay direcciones que apuntan al firmware de la BIOS. Esto incluye:

• El control básico del teclado (para que puedas apretar F2 o Supr y entrar al menú de la BIOS).

• El control básico de la pantalla (para mostrar las letras blancas con fondo negro del inicio).

• El control básico del disco o la disquetera (para poder leer los primeros sectores del disco y buscar el sistema operativo).
• (Ojo: El mouse por lo general no tiene firmware en la IVT en esta etapa física en la arquitectura DOS clásica, porque DOS no usaba mouse por defecto).

Momento 2: El Kernel toma el control y "agranda" la IVT
Cuando el Kernel (como el de FreeDOS) se carga en la memoria, toma el mando de la IVT y empieza a agregar, modificar y redirigir esas entradas para meter sus propios servicios, controladores y programas:

1. Mete los servicios del Sistema Operativo: El kernel reemplaza funciones básicas de la BIOS por las suyas. Por ejemplo, la famosa interrupción INT 21h en DOS, que es el catálogo de servicios del kernel (leer un archivo, crear una carpeta, cerrar un programa).
2. Se cargan los Controladores (Drivers) avanzados: Si conectas un mouse, el kernel ejecuta un driver (como MOUSE.COM o CTMOUSE.EXE). Este driver se instala en la RAM, va a la IVT y agrega el servicio del mouse que la BIOS no había puesto.
3. Se quedan los programas pasivos (TSR): Como vimos antes, programas que simulan multitarea se quedan viviendo en la RAM y modifican la IVT para activarse cuando ocurra un evento (como el programa de captura de pantalla).

Bloque de segmento de programa(psp o program segment prefix)

Toda aplicación cargada en memoria es precedida por un espacio 256bits(ver introduccion)

Bloque de control de memoria:

el bloque de memoria entero que el Kernel le da a un programa para que funcione tiene esos tres componentes alineados uno detrás del otro.

[PIEZA 1: El MCB] [PIEZA 2: El PSP] [PIEZA 3: El PROGRAMA]
(Cartelito del Kernel) \--> (DNI del programa) \--> (El juego o código real)
Mide 16 Bytes Mide 256 Bytes Mide lo que necesite

4. El MCB (La frontera): Es el cartel de control que pone el Kernel. Ocupa un espacio diminuto (16 bytes) y sirve para avisar dónde empieza este programa, cuánto mide y dónde termina.
5. El PSP (La cabecera): Es el "DNI" o la hoja de presentación del programa. Ocupa un poco más de espacio (256 bytes) y contiene la información que el sistema operativo necesita saber sobre el programa mientras este corre.
6. El Programa Real (El cuerpo): Aquí es donde finalmente vive el código del juego o la aplicación, junto con las variables y los datos que tú estás usando.

Mecanismo de Interrupciones por software:

En sistemas operativos modernos y multitarea (como Linux o Windows), una "llamada al sistema" (System Call) es un mecanismo muy complejo y protegido. El procesador tiene que cambiar drásticamente de "Modo Usuario" a "Modo Kernel" mediante hardware para que un programa no rompa a los demás.

En FreeDOS, al ser monotarea y no tener protección de memoria, no existe esa barrera de seguridad. Todo corre al mismo nivel.

En lugar de llamadas al sistema complejas, DOS utiliza interrupciones por software. El kernel de FreeDOS se expone al mundo como un catálogo de funciones y se adueña de un número de interrupción famoso: la INT 21h.

Cuando tu programa (por ejemplo, un juego) quiere guardar una partida en el disco duro, no sabe cómo hablar con los cables del disco. Así que hace esto en su código:

7. Pone en una "cajita" (registro AH) el número de la función que quiere (por ejemplo, el número 3Ch que significa "Crear Archivo").

8. Ejecuta la instrucción: INT 21h.
9. El procesador congela un milisegundo tu juego, mira la IVT (el mapa de interrupciones que ya te sabes de memoria), ve que la INT 21h apunta al Kernel, y salta a ejecutar el código del kernel de FreeDOS para crear el archivo.
El Kernel hace la tarea en un milisegundo y usa la instrucción IRET del procesador para volver al juego.

El IVT puede cumpliar la funcion de un administrador de tareas a los fines lo procesos que estan esperando ser

Eliminados de memoria.

Interrupciones mas importantes:

int13 > acceso a bajo nivel y bios
Int10> servicios de video
Int16 > servicios de teclado

Algunas funciones de int21

01h > lectura de carácter por teclado
09h > escritura en pantalla
3Ch > crear archivo
3Dh > abrir archivo
3Fh > leer archivo
40h > escribir en archivo
2Ah > consulta fecha

Aquí esta la idea mas importante:

"""
Un programa cargado en
memoria bajo FreeDOS se encuentra, en todo momento, en uno de dos estados posibles: en ejecución activa, ocupando la totalidad del procesador hasta que finaliza mediante la función 4Ch de la INT 21h y libera su bloque de control de memoria (MCB); o bien, en el caso particular de los programas
TSR descriptos anteriormente, residente en memoria pero inactivo, a la espera de que una interrupción específica transfiera momentáneamente el control a su rutina de manejo.
"""

Si hay una sola tarea no hay sistema de planificación.

GESTION DE MEMORIA

Su complejidad radica en que solo hay 1mb disponible, todo debe trabajar con ese megabyte.

Para superar esta dificultad se establecio categorias de memoria

- Memoria convencional son los primero 640 kilobytes, es la verdadera memoria disponible para aplicaciones;
- Memoria upper: son los restantes 384 kilobytes se encuentran reservados para funciones especificas de hardware como memoria de video, bios, o controladores adicionales
- UMB o bloques de memoria superior, son huecos que quedan luego de la carga del la upper y que suele ser usado para TSR
- Memoria extendida(XMR): una forma acceder a un espacio luego del primer megabyte mediante un controlador especial llamado HIMEMX, es decir, como todo controlador, es el intermediario entre los megas extras y el procesador.
- Memoria Expandida: actua sobre la upper, pero emulando paginación;

SISTEMAS DE ARCHIVOS: (DESARROLLADO EN INTRODUCCION)

INTERPRETE DE COMANDOS(FREECOM)

constituye la interfaz de línea de comandos a través de la cual el usuario
interactúa de manera directa con el sistema operativo, y es responsable de la interpretación y ejecución de los archivos de procesamiento por lotes (batch files, con extensión .BAT). FreeCOM es cargado por el kernel al final de la secuencia de arranque, según la directiva SHELL especificada en el archivo de configuración del sistema, y permanece residente en memoria durante toda la sesión de trabajo.

Selección representativa de utilidades incluidas en FreeDOS
FORMAT Gestión de discos Da formato lógico a una unidad, preparándola para

FAT.

FDISK Gestión de discos Crea, modifica y elimina particiones en un disco duro.
EDIT Edición de texto Editor de texto de pantalla completa para

configuraciones.

XCOPY Gestión de archivos Copia estructuras completas de directorios y

subdirectorios.

MEM Diagnóstico Muestra el estado de la memoria convencional, UMB

y XMS.

CHKDSK Diagnóstico Verifica la integridad lógica del sistema de archivos

FAT.

ATTRIB Gestión de archivos Consulta y modifica atributos (solo lectura, oculto,

sistema).

DEBUG Programación Depurador de bajo nivel para inspeccionar registros y

memoria.

Aclaracion importante:

El procesador solo puede manipular y trabajar con los datos que se encuentran alojados directamente en la memoria principal (RAM). Cuando una instrucción requiere información externa, el sistema operativo (a través de interrupciones y controladores de bajo nivel) ejecuta las rutinas necesarias para acceder al disco físico, transferir esos datos hacia un bloque libre de la memoria RAM y, recién entonces, permitir que el procesador los lea y procese en sus registros internos.

5. Arquitectura x86, registros y memoria

Estructura ensamblador:

Manipulación de memoria:

En modo real, en sistema DOS tenemos acceso solo a 1MB de memoria, no hay paginación, ni multiproceso, ni niveles de privilegio. Se puede controlador todo el procesador.

Segmentos de memoria(física)

Dirección = (Segment X 0x10) + Offset

De lo anterior se desprende que la memoria tiene dos componentes:
A- segmento: cada segmento es de 64kb. Como el offset es de 16 bits, el rango máximo que puede direccionar dentro de un mismo segmento es de \(2^{16} = 65,536\text{ bytes}\) (64 KB).

Tipos de segmentos:

- CS(Code Segment): el puntero apunta a donde están las instrucciones;
- DS(Data Segment): apunta a tus variables;
- SS(Stack Segment): apunta a las pilas;
- ES(Extra Segment): manipulación de cadenas(string) o memoria de videos;

B- desplazamiento: hasta donde ocupa el recurso. Veremos asignaciones como "0100:10D1H", donde tenemos segmento 0100(donde comienza), y desplazamiento de 10D1(hasta donde llega), donde H solo indica hexadecimal.

Registros Globales(del procesador) y Altas y Bajas:
Dentro del propio chip hay almacenamientos ultra-rápidos que utiliza el procesador, y son instrucciones concretas, abrir, cerrar, borrar, leer, mover(acciones). Es decir, los registros pertenecen al CPU.
Los registros tienen 16bits(2 bytes) dos fragmentos de 8bits para operaciones simples.
- AX: es el acumulador(operaciones aritméticas e interrupciones), se divide en:
■ Alta(AH) y Baja(AL) según el tamaño del dato o numero que vamos a usar.

- BX: direccionamiento base o colocar el puntero en un segmento determinado, indicando el inicio de un archivo(BH y BL)
- CX: Está diseñado específicamente para contar repeticiones. Es un bucle o ejecuta un bucle. Se divide en CH o CL.
- DX: (Datos): Operaciones de entrada/salida y multiplicación/división. Se divide en DH y DL.
- SP: o Stack Pointer o puntero de pila(LIFO- last in, first out)
- Los registros que siguen no se pueden fraccionar, y sirven para desplazarse en momoria(mover el puntero)
■ BP: nos permite anclar o apuntar a un lugar de la pila, sin tener que recurrir a un POP para extraer el dato
■ SI: Apunta a la dirección de memoria física donde se encuentran los datos de origen.
■ DI: Apunta a la dirección de memoria física de destino donde quieres guardar un resultado, hacer una copia, o escribir datos nuevos.

- Movimientos y Asignación: La instrucción MOV destino, origen copia datos entre registros o memoria. Las instrucciones PUSH origen y pop destino manipulan la pila LIFO, fundamental para guardar el estado antes de llamar a subrutinas.

- Interrupciones (System Calls): Dado que FreeDOS no tiene una librería estándar abstracta ni llamadas syscall como Linux, te comunicas con el Kernel mediante Interrupciones de Software, principalmente la INT 21h. Cargas un número de función en el registro AH, configuras otros registros como parámetros y llamas a la interrupción. El procesador detiene tu código, salta a la rutina del Kernel de FreeDOS, y retorna cuando termina.

7. Casos de uso, vigencia y conclusiones

CAPÍTULO 8. CASOS DE USO CONTEMPORÁNEOS Y VIGENCIA DEL PROYECTO

A más de tres décadas de su fundación, resulta legítimo preguntarse qué justifica la continuidad de un sistema operativo diseñado originalmente para procesadores y arquitecturas de cómputo hoy ampliamente superadas. La respuesta se encuentra en un conjunto de nichos técnicos específicos en los cuales las propias características de FreeDOS -ausencia de multitarea, acceso directo al hardware, reducidísimo consumo de recursos- se convierten en ventajas determinantes.

8.1 Actualización de firmware y BIOS/UEFI

Uno de los casos de uso más extendidos de FreeDOS en la actualidad corresponde a su empleo como entorno de arranque para herramientas de actualización de firmware (flashing) de placas base, tarjetas controladoras, unidades de almacenamiento y otros dispositivos. Numerosos fabricantes de equipos industriales y servidores continúan distribuyendo sus utilidades de actualización de BIOS o UEFI como ejecutables diseñados para funcionar bajo un entorno DOS, dado que este ofrece acceso directo y sin intermediarios al hardware. FreeDOS se ha consolidado como el reemplazo libre estándar para esta finalidad, sustituyendo a las versiones antiguas de MS-DOS.

8.2 Ejecución de software heredado (legacy systems)

Un segundo caso de uso relevante corresponde a la ejecución continuada de software industrial, científico o de gestión empresarial desarrollado originalmente para DOS y cuya reescritura resulta inviable. Es el caso de equipamiento de control numérico computarizado (CNC), instrumental de laboratorio o sistemas de punto de venta. FreeDOS permite sustituir una licencia original de MS-DOS por una alternativa libre y gratuita con soporte comunitario.

8.3 Sistemas embebidos y de bajo consumo

Debido a su reducidísimo consumo de recursos de cómputo y de memoria, FreeDOS resulta igualmente atractivo para determinados desarrollos de sistemas embebidos que no requieren las capacidades de un sistema operativo moderno de propósito general, pero sí demandan un entorno de ejecución determinístico, con tiempos de respuesta predecibles y sin la sobrecarga asociada a un planificador de procesos multitarea. En este tipo de aplicaciones, la ausencia de gestión de memoria virtual y de aislamiento de procesos se traduce en un comportamiento más predecible y auditable.

8.4 Preservación digital, retrocómputo y videojuegos clásicos

Finalmente, FreeDOS desempeña un papel relevante dentro de la comunidad de preservación digital y retrocómputo, ofreciendo un entorno libre y legalmente distribuible para ejecutar, ya sea sobre hardware original de época o mediante emuladores, la extensa colección de software y videojuegos originalmente diseñados para MS-DOS entre 1981 y mediados de la década de 1990.
8.5 Desafíos y limitaciones del proyecto en la actualidad
Resulta necesario señalar también las limitaciones y desafíos que enfrenta el proyecto en el año 2026. El primero de ellos es de naturaleza demográfica: al tratarse de un proyecto sostenido por contribución voluntaria, el número de colaboradores activos con conocimiento profundo de programación en modo real y ensamblador x86 resulta reducido, lo cual plantea interrogantes sobre el relevo generacional.

El segundo desafío es de naturaleza técnica: la compatibilidad con hardware contemporáneo resulta cada vez más compleja en la medida en que los fabricantes reducen el soporte de arranque Legacy BIOS en favor de UEFI puro.
El tercer desafío se relaciona con la percepción pública: la asociación casi automática entre FreeDOS y el concepto de obsolescencia informática dificulta que se reconozcan sus casos de uso legítimos e industriales.

8.6 Valor pedagógico
Más allá de sus aplicaciones prácticas directas, FreeDOS conserva un notable valor como recurso pedagógico para la enseñanza de la arquitectura de sistemas operativos a nivel universitario. Su relativa sencillez estructural permite a los estudiantes aproximarse de manera comprensible a conceptos fundamentales -gestión de memoria, interrupciones, estructuras de control de procesos- sin la sobrecarga conceptual de un sistema operativo moderno.
CONCLUSIONES
El estudio realizado a lo largo de este trabajo permite sostener una serie de conclusiones generales respecto de FreeDOS, tanto en su dimensión histórica como en su dimensión estrictamente técnica.
En primer término, FreeDOS constituye un ejemplo particularmente exitoso y duradero de desarrollo de software libre orientado a la preservación de compatibilidad, más que a la innovación funcional. Su valor no reside en superar las capacidades de los sistemas operativos modernos -tarea que nunca formó parte de sus objetivos declarados-, sino en garantizar de manera indefinida, y sin dependencia de una empresa privada, la continuidad de un ecosistema de software y de casos de uso específicos que de otro modo habrían quedado huérfanos de soporte tras la discontinuación comercial de MS-DOS.
En segundo término, el análisis técnico de su arquitectura interna -el modo real de 16 bits, el esquema de segmento:offset, las estructuras PSP y MCB, el mecanismo de interrupciones centrado en la INT 21h, y los distintos esquemas de extensión de memoria (UMB, XMS, EMS)- revela un sistema operativo de notable coherencia interna, cuyas decisiones de diseño responden de manera lógica a las restricciones de hardware de su concepción original.
En tercer término, la incorporación posterior de soporte para FAT32 y direccionamiento LBA demuestra que, pese a su compromiso estricto de compatibilidad binaria con el software heredado, FreeDOS ha evolucionado de manera selectiva para adaptarse a las capacidades del hardware de almacenamiento contemporáneo.
Finalmente, la revisión de sus casos de uso contemporáneos permite concluir que la vigencia de FreeDOS en el año 2026 no obedece a una mera curiosidad histórica o nostálgica, sino a un conjunto identificable de necesidades técnicas reales para las cuales el sistema continúa ofreciendo una solución libre, estable y adecuada.

8. Introducción: virtualización y emulación

Introducción

Diferencias entre Virtualizador y Emulador:

Un virtualizador es un software que permite correr múltiples sistemas operativos (VM) utilizando los mismos recursos de la máquina principal (HOST).

Para que exista virtualización, el procesador debe tener instrucciones de virtualización; según el tipo de procesador pueden ser "Intel VT-x" o "AMD-V", esto permite ejecutar instrucciones sobre el silicio, a

través de una herramienta llamada hipervisor. Estos pueden ser de tipo 1 o 2 según qué tan cerca del Bare-Metal se encuentren. Por ejemplo: VirtualBox es de tipo 2, KVM es de tipo 1 puro integrado en Linux, y QEMU actúa como un emulador/virtualizador que suele combinarse con KVM para lograr un alto rendimiento.

Por su parte un emulador recrea y simula un hardware físico completo, simula un procesador, una memoria, una BIOS, chip de sonido y adaptador de red, todo en una arquitectura diferente. Utiliza técnicas de traducción e interpretación, operando mediante estructuras de datos en memoria (buffers) y lógica de software en lugar de leer archivos de texto plano del disco en cada ciclo. Esto provoca un uso excesivo de recursos, porque tenemos varias capas, la carga en memoria y la traducción constante de cada instrucción al lenguaje del host.

¿Cuándo se usa virtualizador?

- VPS (Hostinger, Digital Ocean) y Cloud (AWS, Azure, Cloudflare)
- Entornos de desarrollo (laboratorios caseros) y DevOps (dockerización de aplicaciones, para testear escalabilidad y alta disponibilidad, acceso y seguridad, pipelines CI/CD)
- Cuando lo que se necesita es la mayor velocidad y rendimiento nativo posible

¿Cuándo se usa emulador?

- Arquitecturas cruzadas: necesitas ejecutar Android en Windows
- Preservación: necesitas ejecutar o testear sistemas antiguos
- Ingeniería inversa y depuración: testear firmware para microcontroladores antes de quemarlo en hardware

9. Práctica: preparar el entorno DOSBox-X

Instrucciones para levantar un emulador usando una imagen(img):

```
# ================================================================================
# BLOQUE NUEVO: Creación física del disco duro virtual (Se ejecuta desde la unidad Z:)
# ================================================================================

# A. Creamos un disco duro virtual de 50MB (formateado automáticamente en FAT)
# Nota: La ruta de salida debe ser accesible por tu sistema operativo anfitrión.
imgmake C:\FreeDOS\freedos.img -t hd -size 50

# ================================================================================
# SCRIPT(Instalación de FreeDOS y configuración de NASM)
# ================================================================================

# 1. Montamos el disco duro virtual recién creado como la unidad C:
# (Eliminamos "-fs fat" para dejar que DOSBox-X lea la geometría real de la partición)
imgmount c C:\FreeDOS\freedos.img -t hdd

# 2. Montamos la carpeta de trabajo principal de Windows como la unidad D:
mount d C:\FreeDOS

# 3. Nos cambiamos a la unidad D: para acceder a los archivos
d:

# 4. Copiamos la estructura de archivos del sistema operativo base a la unidad C:
xcopy d:\freedos c:\freedos /s /e

# 5. Copiamos el kernel principal del sistema directamente a la raíz de la unidad C:
copy d:\kernel.sys c:\

# 6. Copiamos el intérprete de comandos COMMAND.COM a la raíz de la unidad C:
copy d:\command.com c:\

# 7. Cambiamos a la unidad C: para preparar el arranque autónomo
c:

# 8. Escribimos el sector de arranque y transferimos los archivos del sistema al disco virtual
sys c:
```

9. Creamos la carpeta de destino para el ensamblador en el disco virtual
md c:\nasm

10. Regresamos a la unidad D: y entramos a la carpeta de NASM para DOS usando su nombre corto
d:
cd NASM-2~1.05R

```
# 11. Copiamos todos los ejecutables y dependencias (como cwsdpmi) a la unidad C:
xcopy *.* c:\nasm /s /e

# 12. Regresamos al disco virtual C: y configuramos la variable PATH del sistema
c:
set PATH=%PATH%;C:\NASM

# 13. Verificamos la ejecución correcta del compilador nativo
nasm -v
```

10. Práctica: programa con DEBUG

Ejemplo con debug:

```
C:\> debug
-a 100          ; (A)ssemble: Inicia el ensamblador interactivo en el offset 100h.
                  ; Se inicia en 100 porque debug necesita los primeros 256bits(program segment
                  prefix) para:
                  ; 1- parametros que entran por consola
                  ; 2- salida de emergencia del programa
                  ; 3- tablas temporales de archivos(que estamos trabajando en el momemto)
```

xxxx:0100 mov ah, 09 ; [PESA 2 BYTES] (Direcciones 0100h y 0101h).
; 1 byte para el código de operación (opcode) y 1 byte para el valor 09.

xxxx:0102 mov dx, 0109 ; [PESA 3 BYTES] (Direcciones 0102h, 0103h y 0104h).

; 1 byte para el opcode de mover a DX y 2 bytes (1 palabra) para la dirección 0109h.

xxxx:0105 int 21 ; [PESA 2 BYTES] (Direcciones 0105h y 0106h).
; 1 byte para la instrucción INT y 1 byte para el número de interrupción (21).

xxxx:0107 int 20 ; [PESA 2 BYTES] (Direcciones 0107h y 0108h).
; 1 byte para INT y 1 byte para el número de interrupción (20).

xxxx:0109 ; ENTER en línea vacía para salir del modo de ensamblado.

```
-e 0109 'Hola Mundo!$' ; [PESA 12 BYTES] (Desde 0109h hasta 0114h).
            ; Cada carácter y símbolo de texto ocupa exactamente 1 byte en la memoria ASCII.

-r cx         ; (R)egister: Modifica el registro CX (Count) para indicar el tamaño total.
```

CX 0000 ; Valor previo del registro CX.
:15 ; [TOTAL = 21 BYTES EN DECIMAL -> 15h EN HEXADECIMAL]
; Suma exacta:
; 2 bytes (mov ah) + 3 bytes (mov dx) + 2 bytes (int 21) + 2 bytes (int 20)
; = 9 bytes de código.
; + 12 bytes de la cadena 'Hola Mundo!$'
; = 21 bytes decimales.
; Al convertir 21 decimal a base 16 (hexadecimal), da como resultado "15".

```
-n hola.com       ; (N)ame: Asigna el nombre de archivo "hola.com".
-w           ; (W)rite: Escribe en disco los 15h bytes indicados en CX desde el offset 0100h.
Writing 0015 bytes ; Mensaje del sistema confirmando la escritura.
-q           ; (Q)uit: Cierra la utilidad DEBUG.
```

11. Práctica: ensamblador NASM

Codigo en Compilador de Assasmbler:

```
org 100h       ; Directiva para indicar que el código inicia en el offset 100h (.COM)

section .text
```

```
  ; 1. Imprimir el mensaje
  mov ah, 09h ; Función DOS: Imprimir cadena de texto
  mov dx, msg ; Puntero al mensaje
  int 21h      ; Llamada a la interrupción de DOS
```

; 2. Terminar el programa correctamente
mov ah, 4Ch ; Función DOS: Terminar programa con código de salida
mov al, 00h ; Código de salida (0 = Éxito)
int 21h ; Llamada a la interrupción de DOS

```
section .data
  ; El texto, más 0Dh (Retorno de carro), 0Ah (Salto de línea) y '$' (Terminador de DOS)
  msg db 'Hola Mundo!', 0Dh, 0Ah, '$'
```

12. Práctica: OpenGEM

Configurar Interfaz Grafico en freeDOS:

```
REM 1. Montar el disco duro virtual donde vive tu kernel de FreeDOS (D:)
imgmount d C:\FreeDOS\freedos.img -t hdd -fs fat

REM 2. Cambiarse al disco principal D:
d:

REM 3. NOTA: En Windows, debés montar la ISO (FD14BNS.iso) en tu explorador y extraer PACKAGES
\GUI\opengem dentro de C:\FreeDOS\GUI\opengem

REM 4. Desmontar la unidad C: por si está ocupada en DOSBox-X
mount -u c

REM 5. Montar esa carpeta extraída en Windows como la raíz absoluta C: para que OpenGEM no falle
mount c C:\FreeDOS\GUI\opengem

REM 6. Entrar a la unidad C: y lanzar OpenGEM
c:
gem
```